

[Trang chủ](#) > [Bài viết](#) > [Apache Airflow – From Zero to Production](#)

#python



24 phút đọc



738 lượt xem



3 thích



0 bình luận

Apache Airflow – From Zero to Production

AI VIET NAM
@aivietnam.edu.vn

Conquer

[Đăng nhập](#)[Đăng ký](#)

CHƯƠNG 1 — TỔNG QUAN: VAI TRÒ CỦA AIRFLOW TRONG DATA ENGINEERING & MLOPS

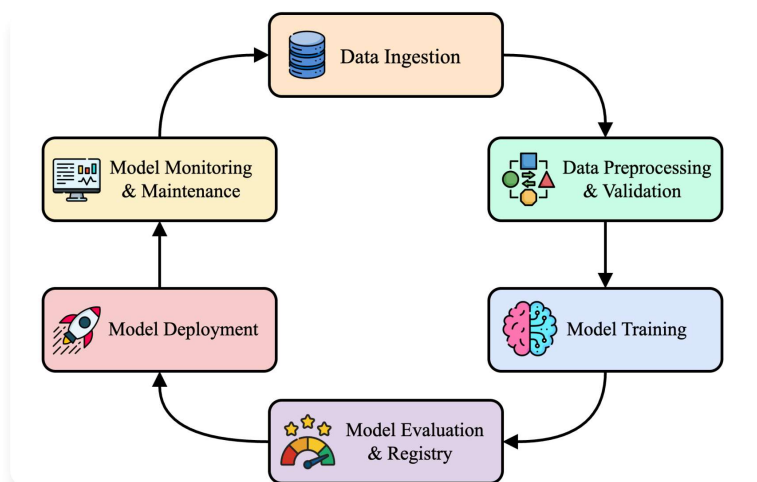
Trong các hệ thống dữ liệu và trí tuệ nhân tạo hiện đại, khối lượng công việc không chỉ nằm ở việc xây dựng một mô hình hay viết một truy vấn dữ liệu. Vấn đề lớn nhất mà doanh nghiệp đối mặt là làm sao để toàn bộ quá trình xử lý dữ liệu, huấn luyện mô hình và triển khai phục vụ người dùng được tự động hóa, nhất quán và có thể lặp lại. Khi chuỗi xử lý trở nên phức tạp—bao gồm thu thập dữ liệu từ nhiều nguồn, chuẩn hóa dữ liệu, chạy nhiều mô hình khác nhau, kiểm tra drift, lưu version, đánh giá và triển khai—việc điều khiển thủ công sẽ nhanh chóng làm hệ thống mất ổn định.

Đây là nền tảng ý nghĩa của Apache Airflow: một hệ thống điều phối (orchestrator) giúp tổ chức, tự động hóa và giám sát toàn bộ các pipeline dữ liệu và machine learning.

1.1. Pipeline trong hệ thống dữ liệu và AI

Trong thực tế, một hệ thống ML/Data bài bản không chỉ có một mô hình duy nhất. Nó là một vòng đời (lifecycle) gồm nhiều bước liên kết chặt chẽ. Có thể mô phỏng

pipeline toàn hệ thống như sau:



Hình 1: Machine Learning Lifecycle Pipeline

Một vòng đời phổ biến bao gồm:

- **Thu thập dữ liệu (Data Ingestion):** lấy dữ liệu từ API, crawler, kho dữ liệu, file log...
- **Tiền xử lý & kiểm định chất lượng (Preprocessing & Validation):** làm sạch, chuẩn hóa, phát hiện dữ liệu hỏng.
- **Huấn luyện mô hình (Training):** lựa chọn thuật toán, version dataset, tuning hyperparameters.
- **Đánh giá & quản lý phiên bản (Evaluation & Registry):** theo dõi hiệu năng, lưu bản mô hình, so sánh phiên bản.
- **Triển khai (Serving):** đưa mô hình vào môi trường inference, quản lý nhiều version cùng lúc.
- **Giám sát (Monitoring):** phát hiện drift, kiểm tra lỗi, theo dõi latency – accuracy.

Điều quan trọng là pipeline này lặp lại liên tục. Một doanh nghiệp có thể phải chạy training mỗi ngày, validate dữ liệu mỗi giờ và kiểm tra drift mỗi 15 phút. Chính sự lặp lại này tạo ra nhu cầu tự động hóa.

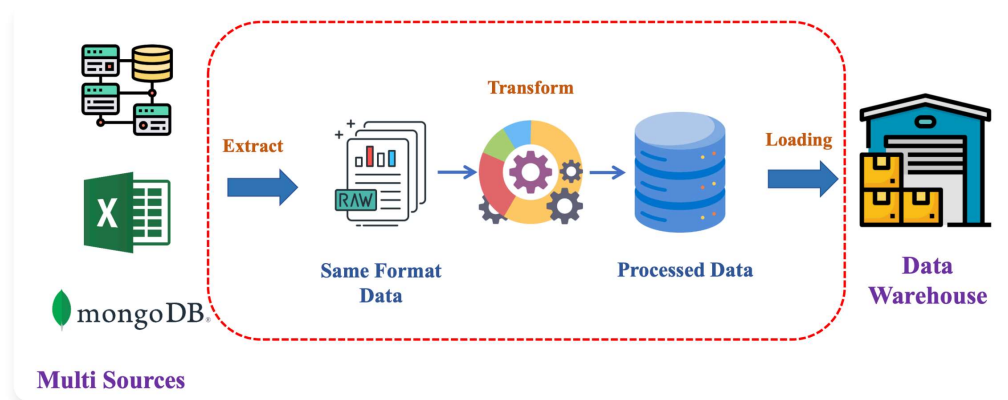
1.2. Data Pipeline – nền tảng của mọi hệ thống hiện đại

Data Pipeline là chuỗi xử lý dữ liệu giúp hệ thống trở nên "có ăn có học". Nếu dữ liệu không ổn định, toàn bộ AI phía sau sẽ lung lay. Một Data Pipeline bài bản

thường bao gồm:

- **Extract** dữ liệu từ nhiều nguồn dị biệt
- **Transform** thành định dạng thống nhất
- **Load** vào Data Warehouse hoặc Data Lake

Các pipeline này thường chạy theo lịch cố định (hàng ngày, hàng giờ), và nhiệm vụ của Airflow là đảm bảo chúng chạy đúng lịch, đúng thứ tự và đúng logic.



Hình 2: Data Pipeline - ETL Process

Airflow thể hiện từng bước của Data Pipeline bằng DAG (Directed Acyclic Graph), giúp người vận hành dễ dàng theo dõi và xử lý sự cố.

1.3. Training Pipeline – xử lý versioning và reproducibility

Trong quá trình huấn luyện mô hình, bạn không chỉ cần mã nguồn và dữ liệu; bạn cần quản lý phiên bản dataset, phiên bản mô hình, các cấu hình huấn luyện (learning rate, batch size...), các thử nghiệm khác nhau. Một Training Pipeline tốt phải:

- Tự động lấy đúng phiên bản dữ liệu
- Chỉ định đúng mô hình hoặc cấu hình cần dùng
- Lưu lịch sử chạy
- Lưu lại artifacts sau khi train
- Tự động chạy lại khi có dữ liệu mới

Airflow đáp ứng đầy đủ: mỗi bước trong training đều được mô tả bằng task rõ ràng. Khả năng "run lại chính xác pipeline cũ" (reproducibility) là yếu tố cốt lõi trong các hệ thống ML quy mô lớn.

1.4. Serving Pipeline – triển khai mô hình đúng phiên bản

Một mô hình triển khai ra môi trường thật không phải là một file .pt hay .pkl đặt bừa trong server. Serving Pipeline cần:

- Chọn đúng phiên bản của mô hình
- Chuyển đổi định dạng cho inference server
- Kiểm định mô hình trước khi deploy
- Triển khai có kiểm soát
- Lưu trạng thái version

Airflow giúp thiết lập một pipeline triển khai rõ ràng: từ việc tải model, đóng gói, chuyển đổi định dạng, cho đến việc đẩy sang server inference.

1.5. Monitoring Pipeline – phát hiện drift và sự cố hệ thống

Một mô hình nếu không được giám sát sẽ nhanh chóng xuống cấp khi phân phối dữ liệu thay đổi. Để đo mức độ thay đổi giữa dữ liệu hiện tại và dữ liệu lúc training, ta dùng một số chỉ số như KL Divergence, một cách đo khoảng cách giữa hai phân phối xác suất:

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \left(\frac{P(i)}{Q(i)} \right)$$

Trong đó:

- $D_{KL}(P \parallel Q)$: Kullback-Leibler Divergence (độ phân kỳ KL) giữa hai phân phối P và Q
- $P(i)$: Xác suất của sự kiện i trong phân phối tham chiếu (thường là phân phối dữ liệu training)
- $Q(i)$: Xác suất của sự kiện i trong phân phối so sánh (thường là phân phối dữ liệu hiện tại/production)

- Giá trị D_{KL} càng lớn, sự khác biệt giữa hai phân phối càng lớn, cho thấy drift càng nghiêm trọng

Airflow có thể đặt lịch chạy việc tính toán drift, lưu kết quả, so sánh ngưỡng và kích hoạt cảnh báo. Ngoài drift, Airflow còn có thể giám sát hiệu năng hệ thống như độ trễ, tỷ lệ lỗi, thời gian chạy task.

1.6. Tại sao Apache Airflow lại quan trọng?

Airflow giải quyết các vấn đề mà mọi hệ thống dữ liệu & AI đều gặp phải:

- Tự động hóa toàn bộ workflows
- Quản lý dependency giữa các bước
- Retry thông minh, không cần viết if-else phức tạp
- Lịch chạy (scheduling) mạnh mẽ
- Quan sát pipeline bằng giao diện UI
- Quản lý version, ghi log chi tiết
- Khả năng mở rộng từ local đến distributed cluster

Nói cách khác, Airflow giúp hệ thống ML/Data vận hành ổn định – có thể tin cậy – có thể tái lập – và dễ mở rộng. Đây là lý do nó trở thành tiêu chuẩn không chính thức trong ngành.

1.7. Ví dụ nhỏ: một DAG đơn giản mô tả pipeline ML

Để kết nối khái niệm với thực tế, ví dụ dưới đây mô tả một DAG rất nhỏ gồm ba bước: ingest dữ liệu → xử lý dữ liệu → huấn luyện mô hình.

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def ingest():
    print("Ingesting data...")

def preprocess():
    print("Processing data...")
```

```
def train():  
    print("Training model...")  
  
with DAG(  
    dag_id="ml_pipeline_example",  
    start_date=datetime(2025, 1, 1),  
    schedule_interval="@daily",  
    catchup=False,  
):  
    t1 = PythonOperator(task_id="ingest",  
python_callable=ingest)  
    t2 = PythonOperator(task_id="preprocess",  
python_callable=preprocess)  
    t3 = PythonOperator(task_id="train_model",  
python_callable=train)  
  
    t1 >> t2 >> t3
```

Đây là một pipeline đơn giản nhưng thể hiện đầy đủ triết lý Airflow: mọi quy trình được biểu diễn bằng DAG, từng bước có quan hệ rõ ràng và hệ thống đảm nhiệm việc orchestration.

CHƯƠNG 2 — KIẾN TRÚC LỖI VÀ CÁC KHÁI NIỆM CƠ BẢN TRONG APACHE AIRFLOW

Để hiểu cách xây dựng một pipeline dữ liệu hoặc pipeline Machine Learning bền vững, ta phải nắm được những thành phần chính trong Airflow: DAG, Task, Operator, Executor, Scheduler, Metadata Database và cơ chế truyền dữ liệu giữa các bước. Những khái niệm này tạo nên nền tảng vận hành của Airflow, giúp người dùng mô hình hóa các công việc phức tạp thành một hệ thống có trật tự, có thể theo dõi, tái lập và mở rộng.

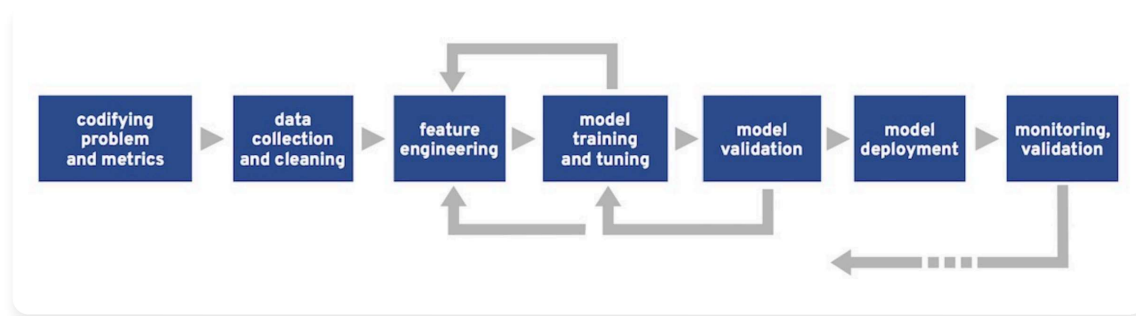
2.1. DAG — Trái tim của Airflow

Một workflow trong Airflow được mô tả dưới dạng DAG (Directed Acyclic Graph) – một đồ thị có hướng và không có chu kỳ. Mỗi đỉnh trong DAG là một task, còn cạnh thể hiện mối quan hệ phụ thuộc giữa các task.

DAG trả lời các câu hỏi:

- Thứ tự thực thi các bước trong pipeline như thế nào?
- Bước nào phải chạy trước bước nào?
- Khi nào có thể chạy song song để tăng tốc?
- Khi xảy ra lỗi, chạy lại bước nào là hợp lý?

DAG đảm bảo rằng quá trình xử lý không tạo ra vòng lặp logic. Điều này ngăn chặn các lỗi nguy hiểm như: mô hình được huấn luyện nhiều lần không kiểm soát, pipeline rơi vào vòng lặp vô hạn, hoặc dữ liệu được xử lý sai thứ tự.



Hình 1: Directed Acyclic Graph - Luồng phụ thuộc giữa các task

2.2. Task — Đơn vị thực thi nhỏ nhất

Trong DAG, mỗi task tương ứng với một hành động cụ thể: đọc dữ liệu, làm sạch dữ liệu, gửi email, chạy lệnh bash, huấn luyện mô hình hoặc lưu log. Airflow không cố gắng "làm tất cả trong một task"; thay vào đó, nó khuyến khích tách workflow thành nhiều task nhỏ, rõ ràng.

Một task trong Airflow có ba đặc điểm chính:

- **Có thể quan sát (observable):** trạng thái thành công/thất bại được lưu lại.
- **Có thể retry:** nếu lỗi tạm thời, task có thể chạy lại theo cấu hình.
- **Có thể mở rộng:** task chạy trên local, container hoặc cluster phân tán tùy executor.

Task là nơi bạn viết logic, DAG là nơi bạn định nghĩa luồng logic.

2.3. Operators — Mẫu định nghĩa hành động

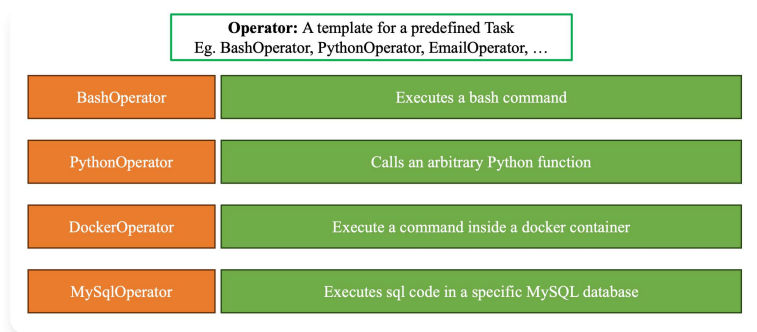
Airflow cung cấp nhiều loại Operator để chuẩn hóa các kiểu task phổ biến. Thay vì viết lại logic kết nối database, gọi API hay chạy file Python, bạn chọn Operator phù

hợp và truyền tham số.

Các loại Operator nổi bật:

- **PythonOperator** — gọi một hàm Python bất kỳ.
- **BashOperator** — chạy lệnh bash.
- **DockerOperator** — chạy tác vụ bên trong container.
- **HTTPOperator** — gọi API, lấy dữ liệu từ endpoint.
- **EmailOperator, SlackAPIOperator** — thông báo sau khi pipeline hoàn thành.

Bản chất Operator là "template", còn Task là instance cụ thể của Operator đó.



Hình 2: Operators Overview

2.4. Executor — Thành phần quyết định task được chạy ở đâu

Nếu DAG là bộ não thì Executor chính là "cơ bắp" của Airflow. Executor quyết định nơi task được chạy và theo phương thức nào.

Các executor nổi bật:

- **LocalExecutor**: chạy nhiều task song song trên một máy.
- **SequentialExecutor**: chỉ chạy một task tại một thời điểm (dùng cho học tập, demo).
- **CeleryExecutor**: phân tán, cho phép hệ thống mở rộng sang nhiều worker.
- **KubernetesExecutor**: mỗi task chạy trong một Pod — phù hợp cho MLOps trên cloud-native.

Sự tách biệt "DAG logic" và "Executor runtime" giúp Airflow triển khai linh hoạt từ local → doanh nghiệp → cluster phân tán.

2.5. Control Flow — Điều khiển luồng xử lý

Sau khi tạo task, ta phải định nghĩa xem chúng liên kết với nhau như thế nào. Airflow cho phép mô tả control flow bằng các toán tử trực quan:

- `A >> B` nghĩa là B chạy sau A
- `[A, B] >> C` nghĩa là C chỉ chạy khi cả A và B đều hoàn thành
- `TaskGroup` giúp gom các task thành nhóm xử lý logic

Nhờ đó, pipeline trở nên trong suốt, dễ đọc và dễ bảo trì.

2.6. Cơ chế truyền dữ liệu giữa các task

Airflow cung cấp hai cách chính để truyền dữ liệu:

1) XCom — truyền data nhẹ

Dành cho dữ liệu nhỏ như:

- tên file
- trạng thái
- dictionary nhẹ
- đường dẫn lưu trữ

Airflow tự động serialize dữ liệu này và lưu vào metadata database.

Ví dụ:

```
from airflow.decorators import task

@task
def load_data():
    return {"rows": 1200, "path": "/data/temp.csv"}

@task
def process(meta):
    print("Rows:", meta["rows"])

process(load_data())
```

2) External Storage — truyền data lớn

Khi dữ liệu lớn (CSV hàng trăm MB, JSON lớn, embedding vector, artifacts), ta lưu vào:

- S3
- MinIO
- GCS
- Database
- Local/SFTP

Và truyền đường dẫn qua XCom.

Đây chính là mô hình tách compute và storage — phù hợp với các hệ thống ML quy mô lớn.

2.7. Kiến trúc tổng thể của Airflow

Airflow vận hành dựa trên 4 thành phần cốt lõi:

- **Scheduler:** đọc DAG, quyết định task nào cần chạy.
- **Executor + Worker:** thực thi task.
- **Webserver (UI):** giao diện quan sát.
- **Metadata Database:** lưu trạng thái DAG, task, log, XCom.

Cấu trúc này đảm bảo:

- Dễ mở rộng
- Có thể quan sát đầy đủ
- Nhiều người dùng truy cập cùng lúc
- Có khả năng phục hồi khi lỗi

2.8. Ví dụ code minh họa cho Core Concepts

Ví dụ này sử dụng PythonOperator, dependency và truyền dữ liệu bằng XCom:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

def extract():
```

```
return "dataset_v1.csv"    # Trả về tên file -> XCom

def transform(filename):
    print(f"Cleaning data from {filename}")

def train_model():
    print("Training model...")

with DAG(
    dag_id="core_concepts_example",
    start_date=datetime(2025, 1, 1),
    schedule_interval="@daily",
    catchup=False,
) as dag:

    t1 = PythonOperator(task_id="extract",
        python_callable=extract)
    t2 = PythonOperator(task_id="transform",
        python_callable=lambda ti:
            transform(ti.xcom_pull(task_ids="extract")))
    t3 = PythonOperator(task_id="train_model",
        python_callable=train_model)

    t1 >> t2 >> t3
```

Đoạn code trên minh họa rõ:

- DAG bao bọc pipeline
- Task dùng PythonOperator
- `t1` → `t2` → `t3` mô tả dependency
- Dữ liệu được truyền giữa các task qua XCom

CHƯƠNG 3 — TRIỂN KHAI APACHE AIRFLOW: TỪ LOCAL ĐẾN PRODUCTION

Trong những chương trước, ta đã hiểu Airflow vận hành như thế nào ở cấp độ logic: DAG, Task, Operators, Executors và các cơ chế truyền dữ liệu. Tuy nhiên, để một hệ thống Airflow chạy được trong thực tế—dù ở máy cá nhân hay trong môi

trường doanh nghiệp—ta cần hiểu cách triển khai (deployment). Triển khai Airflow không chỉ đơn thuần là "cài đặt" mà là thiết kế môi trường phù hợp cho từng nhu cầu: lập trình, kiểm thử, vận hành sản phẩm hay scaling theo nhu cầu hệ thống. Chương này giúp bạn hiểu toàn bộ vòng đời triển khai Airflow từ local đến cluster phân tán.

3.1. Airflow trên môi trường local – nền tảng cho học tập và phát triển

Triển khai Airflow trong môi trường local mang tính "phát triển" hơn là "vận hành". Mục tiêu là:

- Dễ cài đặt
- Dễ gỡ lỗi
- Dễ sửa DAG
- Không yêu cầu scaling

Cách phổ biến nhất là dùng Docker Compose, vì nó đóng gói đầy đủ scheduler, webserver, metadata database và redis vào những container riêng biệt, nhưng vẫn đơn giản và dễ chạy.

Trải nghiệm local bao gồm:

- Web UI truy cập qua localhost
- File DAG sửa là chạy (hot reload)
- Log lưu tại thư mục chia sẻ (./logs)
- Dễ thử nghiệm operator mới
- Dễ thêm plugin, experiment code

Local deployment cho phép lập trình viên xây dựng pipeline mà không sợ phá vỡ môi trường production.

3.2. Cấu trúc thư mục khi chạy Airflow bằng Docker

Trong setup bằng docker-compose, cấu trúc thường bao gồm:

```
project/
|— dags/          # DAG files
|— logs/          # logs từ scheduler & task
```

```
|— plugins/          # custom hooks, operators  
|— config/           # airflow.cfg hoặc config bổ sung  
|— docker-compose.yml
```

Các thư mục này tương ứng với mount point trong container. Đây là cách Airflow tách biệt môi trường chạy và mã nguồn DAG, giúp bạn dễ dàng sửa DAG bằng VSCode nhưng task vẫn chạy trong container.

3.3. Khởi chạy Airflow local bằng Docker Compose

Airflow cung cấp file docker-compose mẫu, bạn chỉ cần tải về và chạy:

```
curl -LfO 'https://airflow.apache.org/docs/apache-airflow/3.1.2/docker-compose.yaml'  
docker compose up -d
```

Sau đó truy cập: <http://localhost:8080>

Tài khoản mặc định thường là:

- Username: **airflow**
- Password: **airflow**

Việc chạy docker-compose như vậy tạo ra một "mini cluster" ngay trên máy bạn: có scheduler, webserver, redis và metadata database PostgreSQL—all-in-one.

3.4. Môi trường phát triển Python (dev env): linting, type checking và hiệu suất

Khi viết DAG, chất lượng code rất quan trọng. Một môi trường Python chuẩn nên có:

- **Conda / venv** để quản lý phiên bản Python
- **Black / isort** để format code
- **mypy** để kiểm tra kiểu tĩnh
- **ruff hoặc flake8** để lint

Điều này giúp DAG rõ ràng, dễ đọc, dễ bảo trì. Trong môi trường doanh nghiệp, gần như bắt buộc phải dùng linting để tránh lỗi trong pipeline production.

3.5. Khác biệt cốt lõi giữa môi trường local và production

Local environment phù hợp cho phát triển. Tuy nhiên, production yêu cầu mức độ ổn định và kiểm soát cao hơn.

Local	Production
Code sửa là chạy	Code được deploy qua Git
Người dùng có toàn quyền	Không ai chỉnh DAG trực tiếp trên server
File DAG lưu trên máy	Webserver không được phép truy cập file hệ thống (chỉ đọc từ metadata DB)
Executor thường là LocalExecutor	Executor là CeleryExecutor hoặc KubernetesExecutor
-	Bảo mật, phân quyền, giám sát chặt chẽ
-	Logging tập trung và persistence

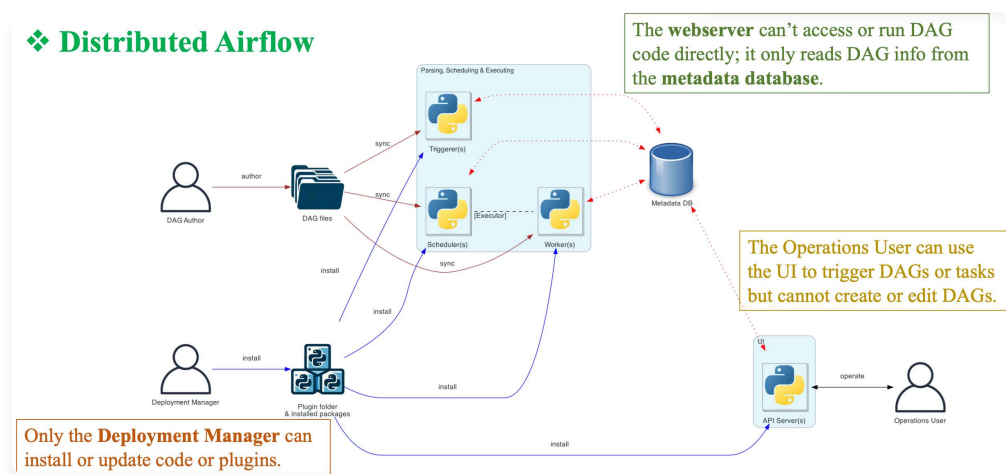
Sự khác biệt này giúp doanh nghiệp đảm bảo rằng hệ thống không bị phá hỏng bởi một thay đổi nhỏ, đồng thời đảm bảo pipeline có thể chạy ổn định nhiều năm.

3.6. Distributed Airflow – mô hình phù hợp cho doanh nghiệp

Trong môi trường lớn, Airflow được triển khai theo mô hình phân tán. Điều này nhằm:

- Tách vai trò dev và ops
- Đảm bảo UI không chạy DAG
- Mã nguồn được deploy có kiểm soát

- Tăng khả năng chịu lỗi
- Chạy nhiều task song song trên nhiều worker



Hình 1: Distributed Airflow Architecture

Ở mô hình này:

- **Webserver**: chỉ hiển thị DAG, không chạy DAG
- **Scheduler**: quyết định task chạy ở đâu
- **Worker**: nơi chạy task thực tế
- **Deployment Manager**: người/tiến trình duy nhất có quyền đưa DAG lên

Đây là mô hình chuẩn trong doanh nghiệp lớn, vì nó hạn chế sai sót và bảo mật tốt hơn.

3.7. Version Control và CI/CD – quy trình triển khai DAG chuyên nghiệp

Các hệ thống production sử dụng Git để kiểm soát mã nguồn Airflow. Một quy trình chuẩn gồm:

1. Developer tạo branch mới và viết DAG
2. Kiểm tra linting, type checking
3. Chạy unit test
4. Merge vào branch staging hoặc production
5. CI/CD tự động deploy DAG vào Airflow environment

Điều này đảm bảo:

- Mọi DAG mới đều được review
- Không deploy nhầm
- Không chạy code lỗi trên production
- Hệ thống ổn định

Airflow phù hợp tuyệt đối với mô hình GitOps.

3.8. IaC (Infrastructure as Code) cho Airflow

Hệ thống Airflow hiện đại thường được triển khai cùng các công cụ IaC như:

- Terraform
- Ansible
- Helm Chart (cho Kubernetes)

IaC cho phép:

- Tự động tạo Airflow cluster
- Tái tạo môi trường chỉ bằng một câu lệnh
- Quản lý version cơ sở hạ tầng
- Giảm lỗi con người

Khi Airflow đi vào giai đoạn doanh nghiệp, IaC gần như là bắt buộc.

3.9. Code mẫu triển khai Airflow Worker bằng CeleryExecutor

Đây là ví dụ đơn giản về cách khai báo CeleryExecutor trong file `airflow.cfg`:

```
executor = CeleryExecutor

[celery]
broker_url = redis://redis:6379/0
result_backend =
db+postgresql://airflow:airflow@postgres:5432/airflow
```

Trong Docker Compose, thêm worker:


```
worker:
  image: apache/airflow:2.7.0
  command: celery worker
  depends_on:
    - scheduler
    - redis
    - postgres
```

Mô hình này giúp mỗi worker chạy task độc lập, mở rộng quy mô dễ dàng.

CHƯƠNG 4 — THỰC HÀNH XÂY DỰNG DATA PIPELINE & TRAINING PIPELINE VỚI AIRFLOW

Nếu ba chương đầu giúp ta hiểu tư duy và kiến trúc Airflow, thì chương này đưa toàn bộ kiến thức vào thực tiễn bằng cách xây dựng một hệ thống pipeline hoàn chỉnh từ đầu đến cuối: thu thập dữ liệu từ API (Scraper DAG), xử lý – lưu trữ – kiểm định – convert – ghi vào database, sau đó tiếp tục huấn luyện mô hình (Training DAG) dựa trên dữ liệu đã được chuẩn hóa. Đây là ví dụ thực chiến mô phỏng cách Airflow vận hành trong các công ty công nghệ: dữ liệu được tự động thu thập, lưu trữ, làm sạch, huấn luyện và cập nhật mô hình định kỳ.

4.1. Tổng quan kiến trúc của hệ thống pipeline thực hành

Toàn bộ pipeline gồm 2 DAG chính:

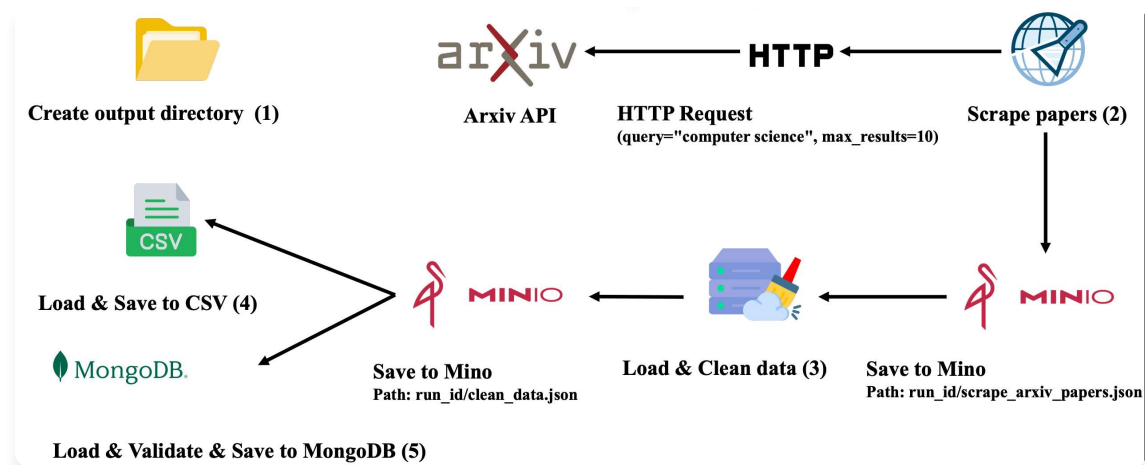
Arxiv Scraper DAG:

- Gọi API Arxiv
- Lưu dữ liệu thô
- Làm sạch
- Lưu vào MinIO
- Convert sang CSV
- Validate → Lưu vào MongoDB

Arxiv Training DAG:

- Kiểm tra dữ liệu có sẵn
- Load từ MongoDB
- Xử lý văn bản
- Chia train/val/test
- Train model
- Evaluate
- Lưu artifact

Dữ liệu được lưu theo cấu trúc thư mục dạng `run_id/task_id.json`, giúp mỗi lần chạy DAG đều có phiên bản dữ liệu riêng—phục vụ việc kiểm tra, tái tạo và debugging.



Hình 1: Kiến trúc Scraper Pipeline

4.2. Arxiv Scraper DAG — Xây dựng ETL pipeline hoàn chỉnh

Scraper DAG đóng vai trò điểm bắt đầu của hệ thống: lấy dữ liệu từ API và đảm bảo dữ liệu đủ sạch để đưa vào database cho training.

DAG bao gồm các bước sau:

4.2.1. Bước 1 — Tạo thư mục output theo run_id

Mỗi lần chạy DAG, ta tạo thư mục mới cho toàn bộ dữ liệu của lần chạy đó.

```
@task
def create_output_dir(run_id: str):
    path = f"/data/{run_id}"
    os.makedirs(path, exist_ok=True)
    return path
```

Ý nghĩa:

- Dễ trace lỗi theo từng run
- Không ghi đè dữ liệu
- Giúp tái hiện pipeline vào bất kỳ thời điểm nào

4.2.2. Bước 2 — Gửi request API Arxiv

Scraper lấy danh sách bài báo theo query (ví dụ: "computer science").

```
@task
def scrape_papers():
    import requests
    url = "http://export.arxiv.org/api/query?
search_query=cs&max_results=10"
    data = requests.get(url).text
    return data
```

Lưu ý thực chiến:

- API Arxiv trả về XML, cần parse → JSON
- Có thể retry khi API timeout
- Thường kết hợp HTTPOperator để quản lý lỗi tốt hơn

4.2.3. Bước 3 — Làm sạch dữ liệu

Dữ liệu Arxiv thường thiếu trường hoặc dư metadata. Ta làm sạch và chuẩn hóa:

```
@task
def clean_data(raw_xml: str):
    parsed = parse_arxiv_xml(raw_xml) # custom function
```

```
cleaned = [normalize(p) for p in parsed]
return cleaned
```

Các bước làm sạch thường bao gồm:

- Chuẩn hóa field title, summary
- Loại bỏ HTML tags
- Chuẩn hóa categories
- Loại duplicate

4.2.4. Bước 4 — Lưu dữ liệu vào MinIO

MinIO đóng vai trò data lake trong bài thực hành.

```
@task
def save_to_minio(data, path, filename):
    import json, minio
    client = minio.Minio("minio:9000", access_key="minio",
secret_key="minio123", secure=False)
    json_bytes = json.dumps(data).encode("utf-8")
    client.put_object("airflow", f"{path}/{filename}",
io.BytesIO(json_bytes), len(json_bytes))
```

Lợi ích:

- Truy cập nhanh
- Mọi DAG đều có thể đọc chung
- Hỗ trợ version theo run_id
- Thích hợp cho dữ liệu vừa và lớn

4.2.5. Bước 5 — Convert sang CSV để dễ phân tích

Một số task training hoặc visualization yêu cầu format CSV.

```
@task
def convert_to_csv(clean_data, output_path):
    df = pd.DataFrame(clean_data)
    csv_path = f"{output_path}/clean_data.csv"
```



```
@task
def check_data_availability():
    count = mongo.count_documents()
    if count == 0:
        raise ValueError("No data available.")
```

4.3.2. Bước 2 — Load dữ liệu từ MongoDB

```
@task
def load_data():
    docs = mongo.find_all()
    return [{"title": d["title"], "abstract": d["summary"],
            "category": d["categories"]} for d in docs]
```

4.3.3. Bước 3 — Tiền xử lý văn bản

Bao gồm:

- lowercase
- remove HTML
- remove stopwords
- tokenize

```
@task
def preprocess_text(data):
    return [preprocess(d["abstract"]) for d in data]
```

4.3.4. Bước 4 — Train/Val/Test Split

```
@task
def split_dataset(data):
```

```
train, val, test = split(data, ratios=[0.7, 0.15, 0.15])  
return train, val, test
```

4.3.5. Bước 5 — Train Model

Model có thể là:

- Logistic Regression
- Naive Bayes
- Small Transformer
- TF-IDF + classifier

```
@task  
def train_model(train, val):  
    vectorizer = TfidfVectorizer()  
    X_train = vectorizer.fit_transform([d["text"] for d in  
train])  
    y_train = [d["label"] for d in train]  
  
    model = LogisticRegression()  
    model.fit(X_train, y_train)  
    return model, vectorizer
```

4.3.6. Bước 6 — Evaluate và lưu artifact

```
@task  
def evaluate_and_save(model, vectorizer, test):  
    acc = evaluate(model, vectorizer, test)  
    save_artifact(model, "model.pkl")  
    save_artifact(vectorizer, "tfidf.pkl")  
    return acc
```

Artifacts bao gồm:

- Model
- Vectorizer

- Tokenizer
- Metrics

Lưu trữ theo run_id giúp lịch sử mô hình minh bạch.

4.4. Trigger, Logs và Debug trong Airflow

Airflow UI cho phép:

- Trigger DAG thủ công
- Theo dõi Tree View của từng task
- Xem log từng lần chạy
- Re-run task nếu cần

Điều này giúp pipeline dễ giám sát và tinh chỉnh liên tục.

KẾT LUẬN — AIRFLOW VÀ VAI TRÒ TRONG HỆ SINH THÁI DATA & MLOPS HIỆN ĐẠI

Apache Airflow không chỉ là một công cụ lập lịch hay một nền tảng chạy tác vụ; nó là bộ khung vận hành (operational backbone) giúp hệ thống dữ liệu và trí tuệ nhân tạo duy trì sự ổn định, khả năng mở rộng và tính tái lập. Trong bối cảnh doanh nghiệp ngày càng phụ thuộc vào dữ liệu thời gian thực và mô hình AI có tuổi thọ ngày càng ngắn hơn, Airflow đóng vai trò điều phối mọi hoạt động: từ thu thập dữ liệu, xử lý, huấn luyện mô hình, triển khai, đến giám sát toàn bộ vòng đời.

Tổng kết các chương

Chương 1 — Ta nhìn thấy cách Airflow giải quyết vấn đề ở tầm hệ thống: quản lý Data Pipeline, Training Pipeline, Serving Pipeline và Monitoring Pipeline một cách tổng thể. Airflow đảm bảo mọi quy trình diễn ra đúng thứ tự, có khả năng khôi phục khi lỗi, và được ghi lại đầy đủ để kiểm tra và tái tạo.

Chương 2 — Tập trung vào lõi kiến trúc Airflow — DAG, Task, Operator, Executor, XCom và Metadata Database — cho thấy Airflow không chỉ mạnh ở khả năng điều phối mà còn ở tính linh hoạt. Pipeline có thể được mô tả như một đồ thị logic rõ ràng, vận hành bởi một kiến trúc tách biệt giữa logic (DAG) và compute (Executor). Đây chính là lý do Airflow dễ mở rộng từ môi trường local nhỏ đến cluster doanh nghiệp lớn.

Chương 3 — Ta đi vào triển khai: từ việc khởi chạy Airflow trong local bằng Docker Compose, cho đến mô hình distributed production sử dụng CeleryExecutor hoặc KubernetesExecutor. Airflow chỉ thật sự phát huy giá trị khi được triển khai đúng cách: có phân quyền, có CI/CD, có GitOps và có môi trường production độc lập với development.

Chương 4 — Đưa mọi thứ vào bài thực hành thực tế. Qua hai DAG lớn—Scraper DAG và Training DAG—ta thấy Airflow giúp tự động hóa toàn bộ pipeline dữ liệu và mô hình theo cách minh bạch, modular và dễ kiểm soát. Từ việc lấy dữ liệu từ API Arxiv, lưu vào MinIO, làm sạch, convert, validate, lưu vào MongoDB cho đến việc training, evaluation và lưu artifacts, Airflow cho phép việc xây dựng pipelines trở nên khoa học, chuyên nghiệp và có khả năng mở rộng theo quy mô sản phẩm thật.

Tầm quan trọng của Airflow

Apache Airflow đã trở thành một thành phần không thể thiếu trong hệ sinh thái Data Engineering và MLOps hiện đại. Với khả năng tự động hóa, quản lý dependency, retry logic, logging và monitoring, Airflow giúp các tổ chức xây dựng và vận hành các hệ thống dữ liệu và AI một cách ổn định, tin cậy và có thể mở rộng.

Tags: #python

Chia sẻ:    

Bài viết liên quan

Project 6.1: Dự đoán giá cổ phiếu với mô hình kết hợp DLinear và Transformer

Project 6.1 đề xuất mô hình Hybrid DLinear-Transformer trong bài toán dự báo giá cổ phiếu FPT 100 ngày tới. Mô hình này tách biệt Xu hướng (Trend) để qua mô hình qua mô hình Lineare và...

Nguyễn Minh Quân

15/12/2025

Module 6 - Project 6.1: Báo cáo Tóm tắt Thử thách Linear Forecasting Challenge (Dự báo giá cổ phiếu)

Báo cáo sơ lược về dự án cuối Module 6 của nhóm CONQ008.

Nguyễn Ngọc Dung

15/12/2025

Hồi Quy Softmax: Nền Tảng Cho Phân Loại Đa Lớp Trong Deep Learning

Hồi quy Softmax là kỹ thuật quan trọng trong phân loại đa lớp, giúp mô hình chuyển logits thành xác suất chuẩn hóa và dự đoán chính xác lớp đầu ra. Kết hợp với hàm mất mát entropy chéo và...

Nguyễn Tuấn Minh

15/12/2025

Bình luận (0)



Đăng nhập để tham gia thảo luận

 Đăng nhập



Chưa có bình luận nào. Hãy là người đầu tiên!



AIO Conquer

Internal Blog Platform for AIO



LIÊN KẾT

[Trang chủ](#)

[Bài viết](#)

[Giới thiệu](#)

[Công thức toán](#)

[Liên hệ](#)

HỖ TRỢ

[Trung tâm trợ giúp](#)

[Điều khoản](#)

[Bảo mật](#)

[Góp ý](#)

© 2025 AIO Conquer Blog. All rights reserved.

Made with by



AI VIET NAM
@aivietnam.edu.vn